

Business Requirements Document (BRD)

Provider Performance Analytics Dashboard (POC)

The Provider Performance Analytics Dashboard is an enterprise-level Proof of Concept (POC) application designed to monitor provider performance using KPI metrics, score tracking, analytics dashboards, risk identification, authentication, authorization, logging, and automated testing.

Project Objective

Build a modern analytics dashboard to track provider performance using KPI cards, provider trends, at-risk identification, score management, and reporting.

Technology Stack

Frontend: React
Backend: .NET Web API
Database: SQL Server
ORM: Entity Framework Core
Authentication: JWT Authentication
Authorization: Role-Based Access Control (RBAC)
Automation: Playwright with TypeScript
Logging: Serilog
API Documentation: Swagger/OpenAPI

Key Functional Features

- Login and Authentication
- Role-Based Authorization
- Dashboard KPI Cards
- Provider Management Module
- Provider Score Tracking
- Search, Filter, Sorting, Pagination
- Provider Detail View
- CRUD Operations
- Dashboard Charts and Analytics
- API Validation and UI Validation
- Error Handling and Logging
- Audit Logging
- Responsive UI

Authentication & Authorization

The system will implement JWT-based authentication with role-based access control. Users will be categorized as Admin, Manager, and Viewer with different access levels. Protected routes and secured APIs will be implemented.

Dashboard Features

KPI Cards:

- Average Score
- Total Providers
- At-Risk Providers
- Monthly Improvement

Dashboard Analytics:

- Monthly score trends
- Risk distribution charts
- Provider performance graphs

Validation Rules

- Provider name is mandatory
- Score must be between 0 and 5
- Duplicate providers are not allowed
- Mandatory field validation in UI and API

Database Tables

- Users
- Roles
- Providers
- ProviderScores
- ErrorLogs
- AuditLogs

Architecture

React Frontend → React Query/Axios → .NET Web API → Middleware → Service Layer → Repository Layer → Entity Framework Core → SQL Server

Advanced Technical Enhancements

- Global Exception Handling Middleware
- Repository Pattern
- Service Layer Architecture
- DTO Pattern
- AutoMapper
- FluentValidation
- Serilog Structured Logging
- Soft Delete Functionality
- API Versioning
- Dashboard Performance Optimization
- React Memoization and Lazy Loading
- API Caching using React Query

Logging & Monitoring

Centralized logging will be implemented using Serilog. Application errors, API exceptions, authentication failures, and database errors will be logged into database and log files.

Audit Logging

The system will maintain audit records for Create, Update, and Delete operations. Audit logs will include: • Action Type • Old Value • New Value • Modified By • Timestamp

Security Features

- JWT Token Authentication
- Password Hashing
- HTTPS Enforcement
- Secure API Endpoints
- CORS Policy Configuration
- Protected Frontend Routes

Playwright Automation

Automated testing will be implemented using Playwright with TypeScript. Test Coverage: • Login Testing • Dashboard Validation • Provider CRUD Operations • Validation Testing • Search & Filter Testing • UI Error Handling

Non-Functional Requirements

- Responsive UI Design
- Scalable Architecture
- High Maintainability
- Secure API Communication
- Fast Dashboard Performance
- Centralized Error Handling

Documentation Deliverables

- BRD Document
- Architecture Document
- API Documentation
- Database ER Diagram
- README Setup Guide
- Playwright Test Report
- Demo Script

Future Enhancements

- Real-time dashboard updates using SignalR
- AI-based risk prediction
- Email notifications

- Export reports to PDF/Excel
- Advanced analytics dashboard

Conclusion

This POC demonstrates enterprise-level full-stack development practices using React, .NET Web API, SQL Server, and Playwright automation. The project showcases authentication, authorization, layered architecture, validation, logging, dashboard analytics, and automated testing suitable for a 3+ years experienced developer portfolio.